Desarrollo de aplicaciones seguras

MÁSTER UNIVERSITARIO EN SEGURIDAD INFORMÁTICA (CIBERSEGURIDAD)

Sara Balderas Díaz (sara.balderas@uca.es)
Gabriel Guerrero Contreras (gabriel.guerrero@uca.es)

CONSTRUCCIÓN DE APLICACIONES SEGURAS (PARTE 1)

Por qué importa: aplicaciones bajo presión

>311.000

millones de ataques web y API observados en 2024

+33 %

incremento interanual de ataques

Una aplicación expuesta recibe entradas que su equipo no controla.

La seguridad debe formar parte de los **requisitos**, del código y de las **pruebas**.

¿Qué diferencia hay entre un intento de ataque y una intrusión?

[4] Akamai, State of Apps and API Security 2025. Datos de 2024.

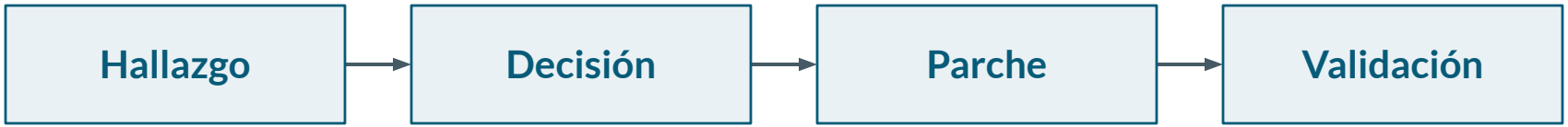
Encontrar fallos no basta, hay que corregirlos

+47 %

en el tiempo medio de corrección desde 2020

El trabajo técnico continúa después de la alerta.

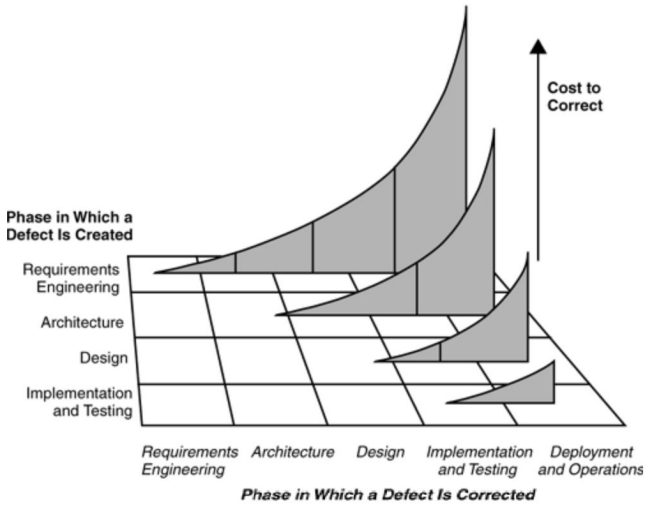
Hay que **confirmar la causa**, decidir la prioridad y **verificar el cambio**.



¿En qué paso puede atascarse un hallazgo que ya conocemos?

[5] Veracode, State of Software Security 2025.

Construir seguridad desde el principio



Reglas de ingeniería para el coste de eliminación de defectos (Mead et al., 2004).

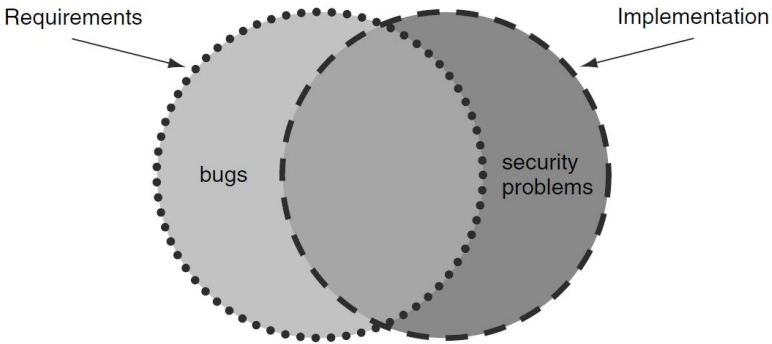
EJEMPLO

ReservaLab (aplicación de ejemplo) permite reservar equipos, consultar informes y descargar resultados.

Usuarios: alumnado, responsables de laboratorio y administración.

Funcionalidad de **descargar un informe** requiere la gestión de permisos. Corregirlo al final exige cambiar API, pruebas y **datos** de **permisos**.

Probar también lo que debe rechazarse



Las pruebas que corroboran que la implementación cumple con la especificación no tendrán en cuenta muchos problemas de seguridad (Chess et al., 2007).

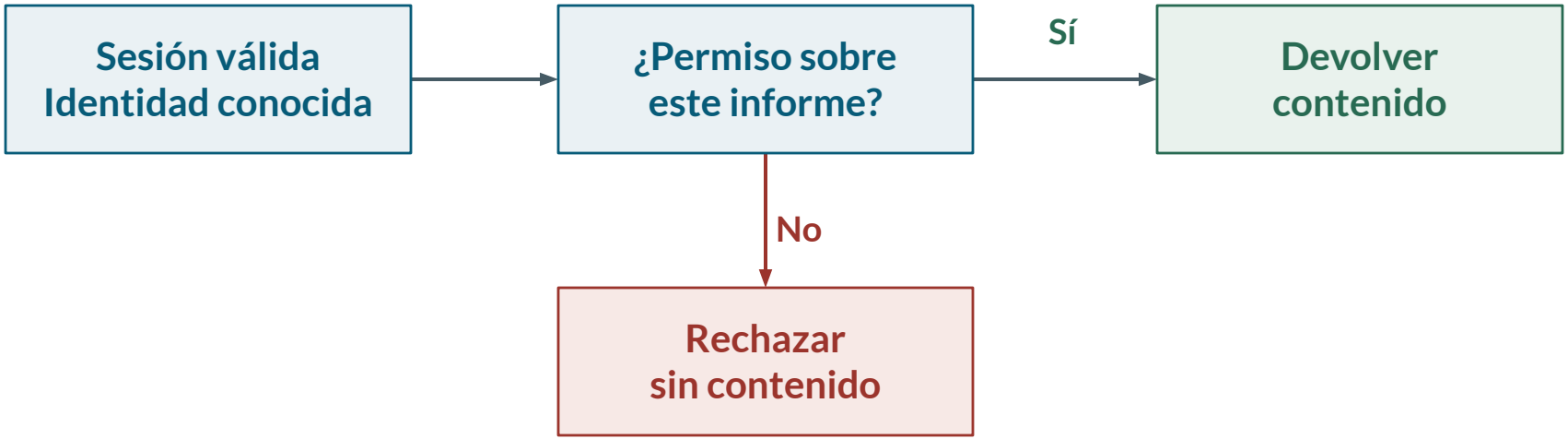
EJEMPLO
Una **prueba funcional verifica una reserva válida**. Una **prueba de seguridad comprueba un uso que debe impedirse**.

Caso normal: un estudiante consulta su informe.

Caso de abuso: intenta descargar el de otro con su sesión.

Prueba positiva: obtiene su informe
Prueba negativa: la petición ajena se rechaza sin devolver contenido

Autenticar no es autorizar

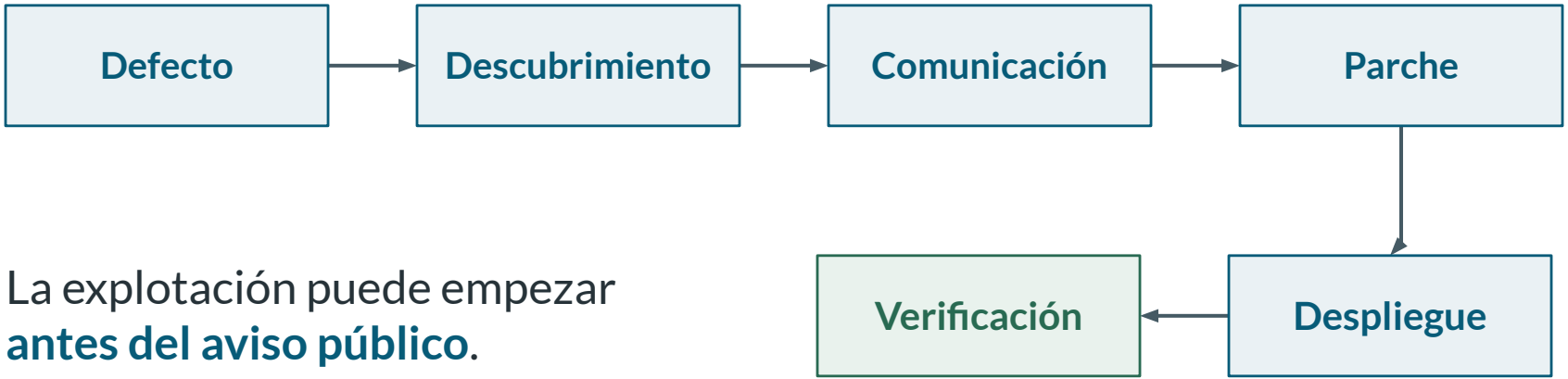


La decisión se aplica en el **servidor**, para cada recurso y operación.

¿Ocultar el botón de descarga resuelve el problema?

El ciclo de una vulnerabilidad

Vulnerabilidad: Debilidad de seguridad que podría ser explotada por un atacante



La explotación puede empezar **antes del aviso público**.

El **seguimiento** confirma qué versiones siguen expuestas

Las **pruebas de regresión** comprueban que un cambio no haya estropeado algo que antes funcionaba

Publicar un parche no cierra el caso

Parche publicado: **ReservaLab 1.4.1**

Laboratorio A
1.4.1
Instala versión que corrige el fallo; debe verificar corrección

Laboratorio B
1.3
Sigue utilizando la versión afectada

Laboratorio C
¿Versión? Desconocida
Hay que averiguar qué versión tiene antes de valorar su exposición

Corregir el repositorio y actualizar la **instalación real** son hitos distintos.

¿Qué evidencia permitiría cerrar la incidencia?

CVE, CWE y severidad

Common Weakness Enumeration (CWE)

Catálogo y taxonomía de **tipos de debilidade**s de software y hardware

Common Vulnerabilities and Exposures (CVE)

Programa y catálogo de **vulnerabilidades concretas** identificadas públicamente

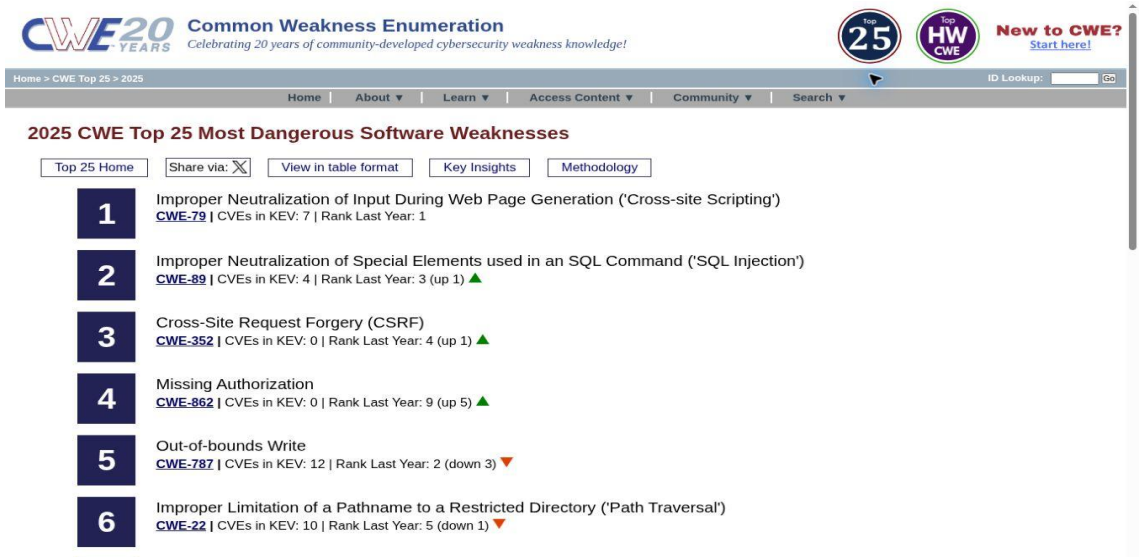
Common Vulnerability Scoring System (CVSS)

Estándar para evaluar la **gravedad de una vulnerabilidad (severidad técnica)** mediante una puntuación

Ejemplo: se descubre que una **aplicación permite descargar documentos ajenos sin comprobar los permisos**. Ese fallo concreto puede recibir un **identificador CVE**.

¿Un CVSS alto implica la misma prioridad en todo sistema?

CWE Top 25: una referencia para elegir casos

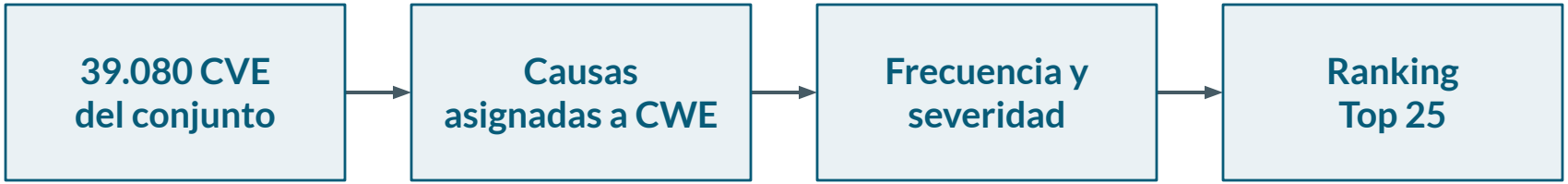


1. **CWE-79**
Cross-Site Scripting
2. **CWE-89**
SQL Injection
4. **CWE-862**
Missing Authorization
6. **CWE-22**
Path Traversal

Ranking determinado por frecuencia y gravedad

[6] MITRE CWE Top 25, edición 2025.

Cómo se construye el ranking

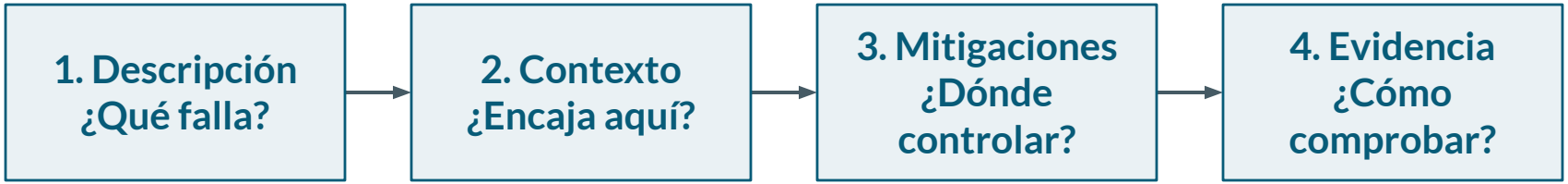


Una lista basada en **vulnerabilidades publicadas** ayuda a priorizar la formación.

La aplicación concreta requiere además **contexto y pruebas**.

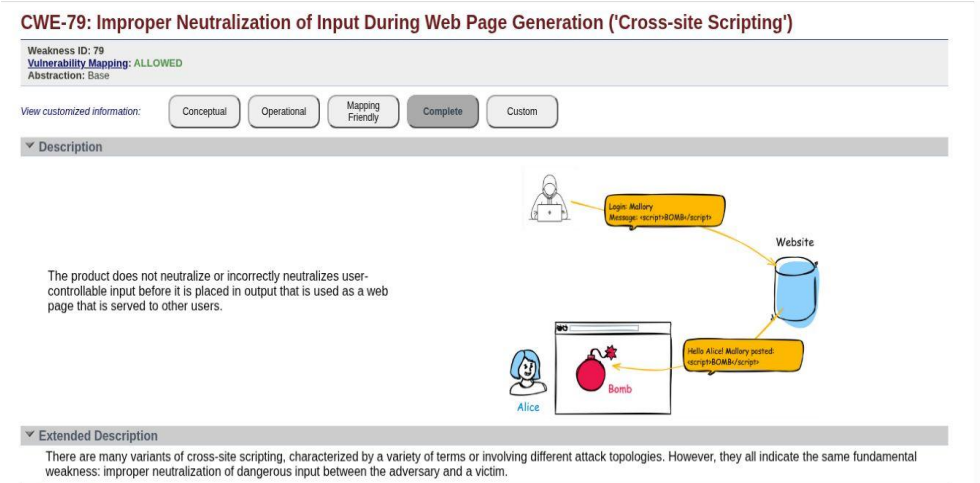
MITRE, metodología del Top 25 de 2025. CVE publicados: 01/06/2024–01/06/2025.

Una ficha CWE se lee buscando una causa



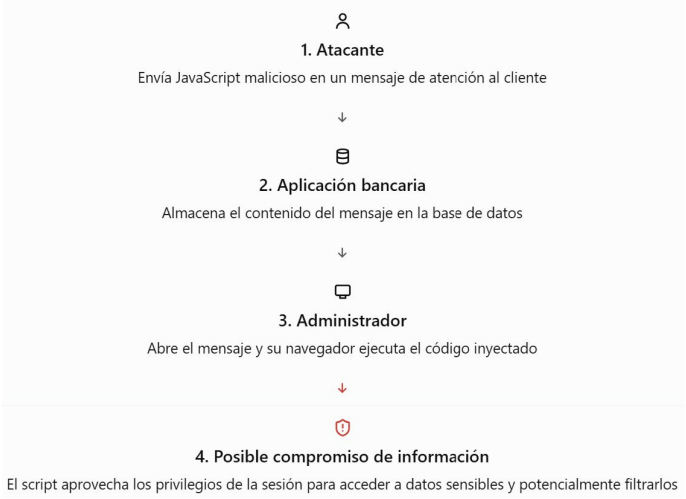
https://cwe.mitre.org/about/new_to_cwe.html

CWE-79: el navegador interpreta un dato (i)



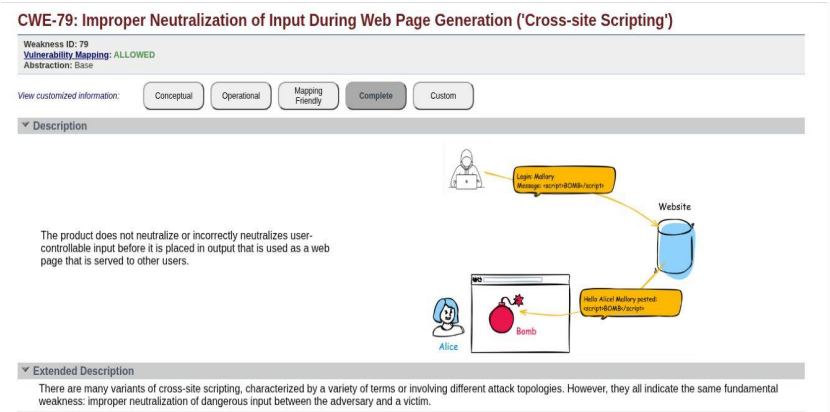
Causa: Una entrada llega a un contexto ejecutable.

EJEMPLO: un atacante compromete el panel de administración de un banco.



MITRE CWE-79.

CWE-79: el navegador interpreta un dato (ii)



1. Código vulnerable: El panel de administración de un banco muestra los mensajes enviados por los clientes (¿atacante?).

```
//código JavaScript
panel.innerHTML = mensajeCliente;
```

Problema: el navegador interpreta el contenido del mensaje como HTML ejecutable.

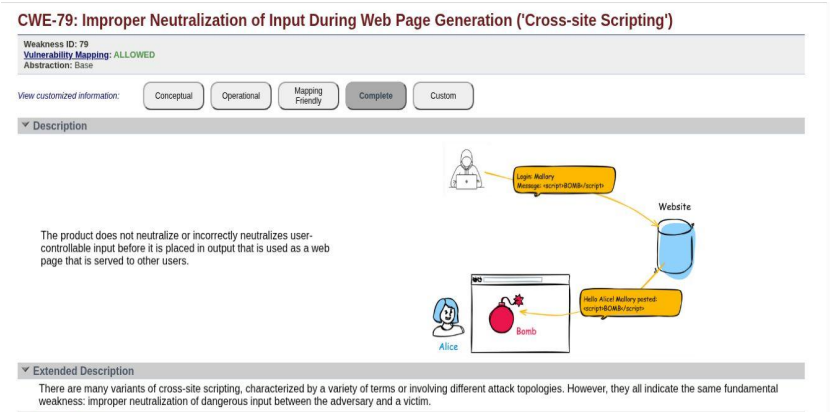
2. Ataque: El atacante envía este mensaje.

```
<!-- código HTML -->
<img src=x
onerror="fetch('/api/admin/transferencias/123
/aprobar',{method:'POST'})">
```

El navegador ejecuta el JavaScript e intenta aprobar una transferencia utilizando su sesión autenticada (*cookies*).

MITRE CWE-79.

CWE-79: el navegador interpreta un dato (iii)



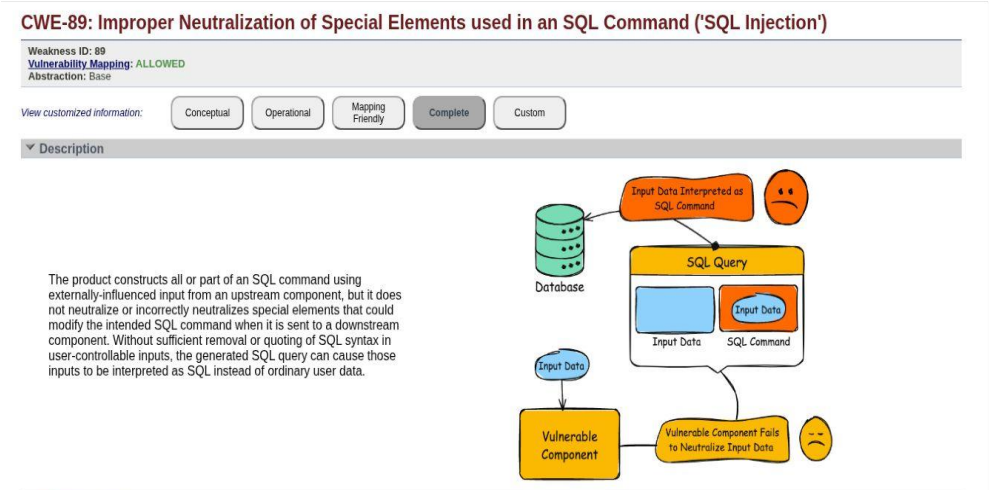
3. Código corregido

```
//código JavaScript
panel.textContent = mensajeCliente;
```

El mensaje se muestra como texto, sin ejecutar el código inyectado.

MITRE CWE-79.

CWE-89: un dato altera la estructura SQL (i)



Causa: La entrada modifica la consulta.

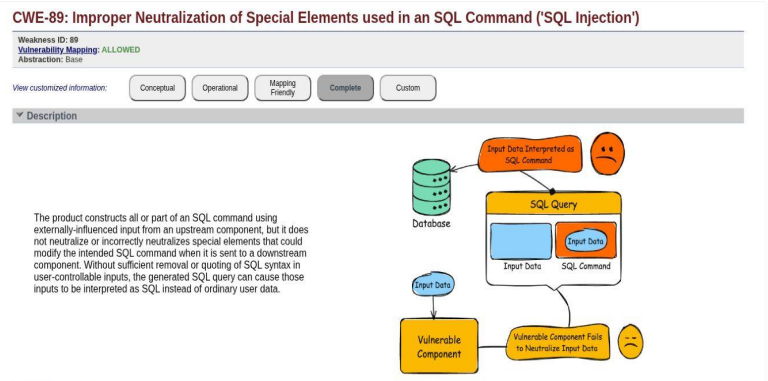
EJEMPLO: acceso a cuentas bancarias de otros clientes. Un usuario autenticado debería poder solo consultar sus propias cuentas.

```
#código Python
cuenta = request.args["cuenta"]
usuario_id = current_user.id

query = f"""
SELECT id, titular, iban, saldo
FROM cuentas
WHERE usuario_id = {usuario_id} AND id
= {cuenta}
"""
db.execute(query)
```

MITRE CWE-89.

CWE-89: un dato altera la estructura SQL (ii)

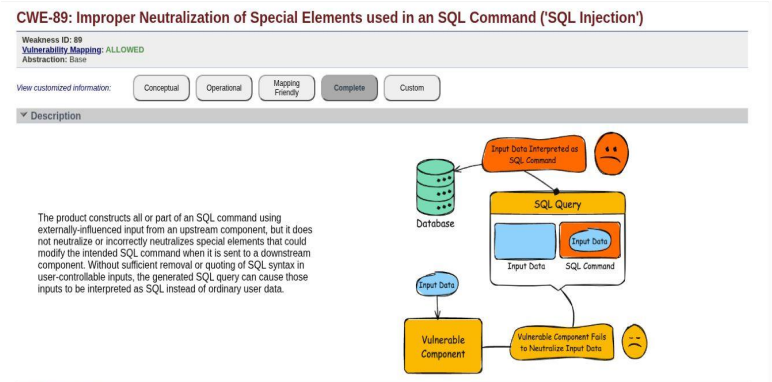


Funcionamiento normal: usuario 42 consulta su cuenta 100

```
– consulta SQL
SELECT id, titular, iban, saldo
FROM cuentas
WHERE usuario_id = 42 AND id = 100;
```

MITRE CWE-89.

CWE-89: un dato altera la estructura SQL (iii)



Funcionamiento normal: usuario 42 consulta su cuenta 100

```
– consulta SQL
SELECT id, titular, iban, saldo
FROM cuentas
WHERE usuario_id = 42 AND id = 100;
```

Funcionamiento inadecuado: atacante manipula la entrada

En lugar de un identificador introduce: `0 OR 1=1`

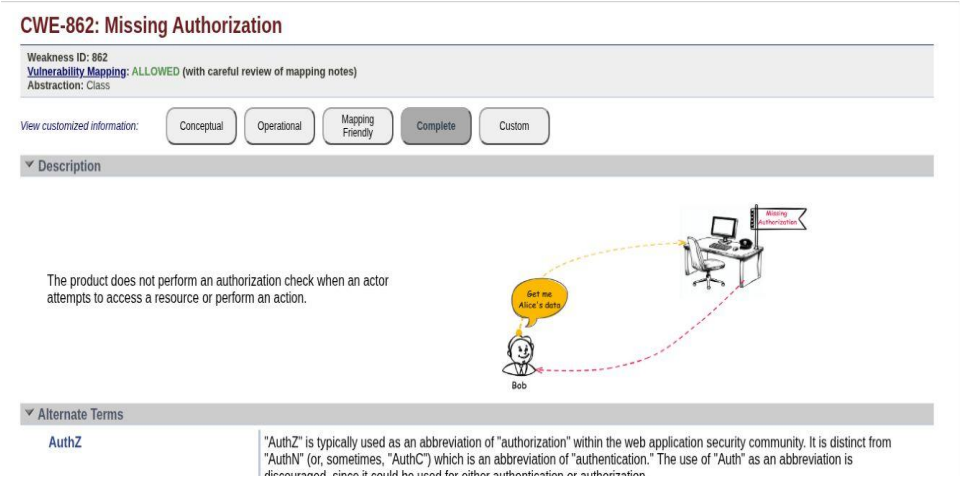
```
– consulta SQL
SELECT id, titular, iban, saldo
FROM cuentas
WHERE usuario_id = 42 AND id = 0 OR 1=1;
```

que equivale a:

```
– consulta SQL
WHERE (usuario_id = 42 AND id = 0) OR 1=1
```

MITRE CWE-89.

CWE-862: falta la decisión de autorización (i)



CWE-862: Missing Authorization

Weakness ID: 862
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Class

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product does not perform an authorization check when an actor attempts to access a resource or perform an action.

Alternate Terms

AuthZ

"AuthZ" is typically used as an abbreviation of "authorization" within the web application security community. It is distinct from "AuthN" (or, sometimes, "AuthC") which is an abbreviation of "authentication." The use of "Auth" as an abbreviation is discouraged, since it could be used for either authentication or authorization.

Causa: La aplicación ejecuta una operación protegida sin comprobar si el usuario tiene permisos para realizarla

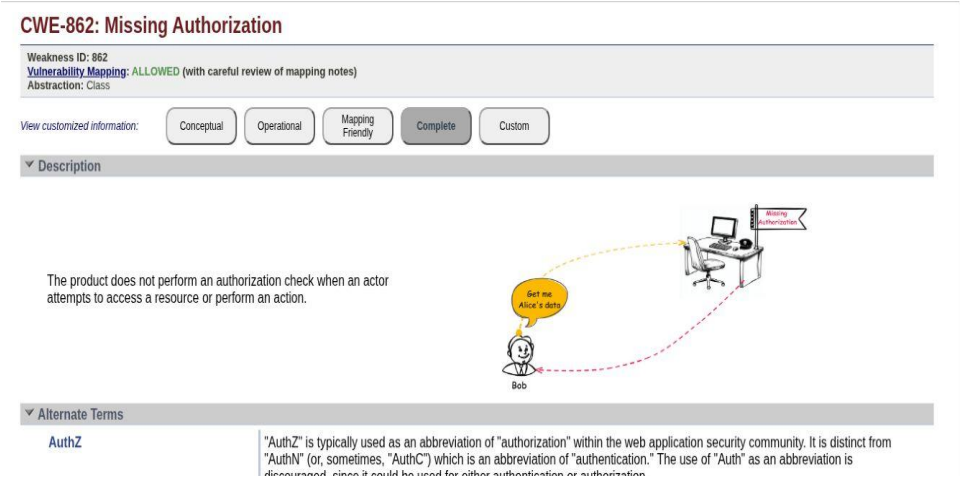
EJEMPLO: cualquier empleado puede descargar las nóminas de toda la plantilla.

```
#código Python
@app.route("/nominas/exportar")
@login_required
def exportar_nominas():
    return generar_csv_todas_nominas()
```

La aplicación comprueba que el usuario ha iniciado sesión (`@login_required`), pero no comprueba si está autorizado para exportar las nóminas.

MITRE CWE-862.

CWE-862: falta la decisión de autorización (ii)



CWE-862: Missing Authorization

Weakness ID: 862
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Class

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product does not perform an authorization check when an actor attempts to access a resource or perform an action.

Alternate Terms

AuthZ

"AuthZ" is typically used as an abbreviation of "authorization" within the web application security community. It is distinct from "AuthN" (or, sometimes, "AuthC") which is an abbreviation of "authentication." The use of "Auth" as an abbreviation is

Posible solución:

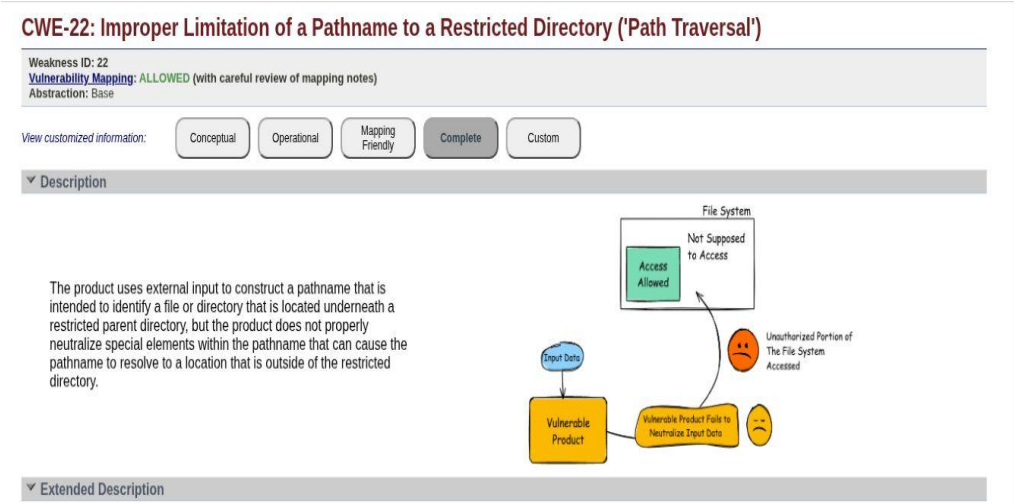
```
#código Python
@app.route("/nominas/exportar")
@login_required
def exportar_nominas():

    if current_user.role != "rrhh":
        abort(403)

    return generar_csv_todas_nominas()
```

MITRE CWE-862.

CWE-22: el archivo sale del directorio previsto



CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

Weakness ID: 22
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product uses external input to construct a pathname that is intended to identify a file or directory that is located underneath a restricted parent directory, but the product does not properly neutralize special elements within the pathname that can cause the pathname to resolve to a location that is outside of the restricted directory.

Extended Description

EJEMPLO: Imagina una aplicación web que permite descargar informes de sensores almacenados en este directorio:

```
/app/
├── reportes/
│   ├── enero.csv
│   └── febrero.csv
└── config/
    └── secretos.txt
```

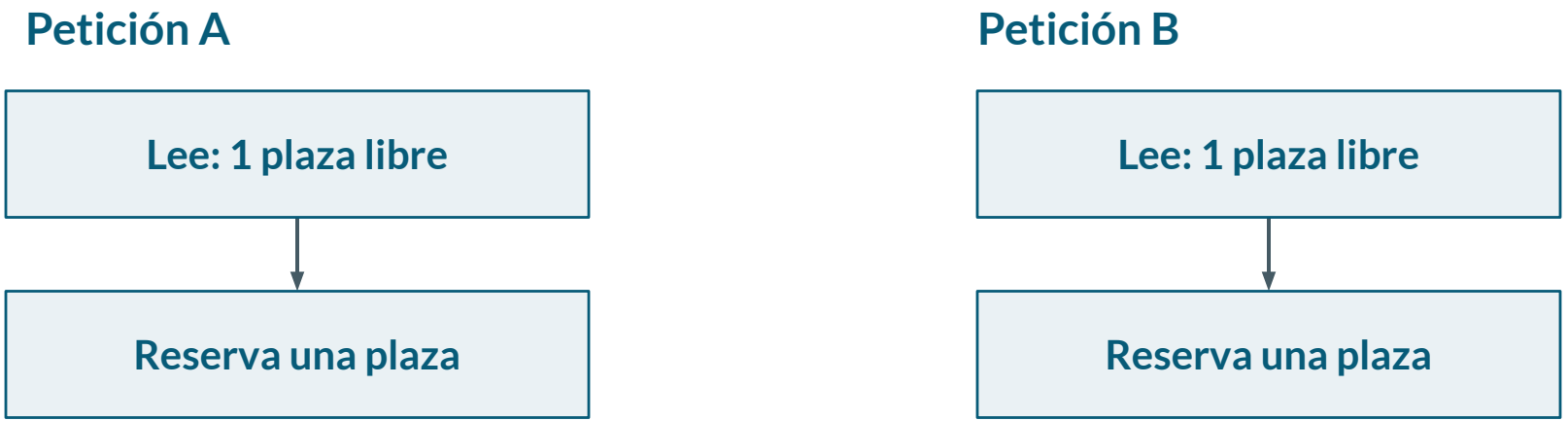
Una petición legítima sería:
`/descargar?archivo=enero.csv`

Pero ¿qué ocurre si el usuario introduce lo siguiente?
`/descargar?archivo=../config/secretos.txt`

MITRE CWE-22.

Operaciones atómicas

Ejemplo: Tiempo y estado. Una plaza, dos peticiones.



Resultado: **2 reservas para 1 plaza**. Comprobar y actualizar deben coordinarse.

Errores: fallar sin revelar ni conceder información (i)

CONCEPTO

Una **excepción debe producir una respuesta controlada y conservar la política de seguridad.**

EJEMPLO / APLICACIÓN

Si falla el servicio de **permisos**, devolver "permitido" tiene consecuencias. Mostrar el error interno puede revelar detalles del sistema.

¿Qué recibe el usuario y qué queda en el registro?

Errores: fallar sin revelar ni conceder información (ii)

EJEMPLO / APLICACIÓN

¿Qué ocurre durante un ataque?

1. Un empleado sin permisos intenta aprobar una transferencia de 250.000 €.
2. El servicio de autorización está temporalmente fuera de servicio.
3. La llamada a `verificar_permisos()` genera un TimeoutError.
4. La aplicación interpreta el error como permitido = True.
5. La transferencia queda aprobada sin haber verificado los permisos del empleado.

```
# Código Python – inadecuado
@app.post("/transferencias/<int:id>/aprobar")
@login_required
def aprobar(id):
    try:
        permitido = verificar_permisos(
            current_user.id, "aprobar", id
        )
    except TimeoutError:
        permitido = True  # VULNERABILIDAD

    if not permitido:
        abort(403)

    aprobar_transferencia(id)
    return {"estado": "aprobada"}
```

Errores: fallar sin revelar ni conceder información (iii)

EJEMPLO / APLICACIÓN

¿Qué ocurre durante un ataque?

1. Un empleado sin permisos intenta aprobar una transferencia de 250.000 €.
2. El servicio de autorización está temporalmente fuera de servicio.
3. La llamada a `verificar_permisos()` genera un TimeoutError.
4. La transferencia no queda aprobada sin haber verificado los permisos del empleado.

```
# Código Python — corregido
@app.post("/transferencias/<int:id>/aprobar")
@login_required
def aprobar(id):
    try:
        permitido = verificar_permisos(
            current_user.id, "aprobar", id
        )
    except TimeoutError:
        app.logger.exception("Fallo de autorización")
        return {"error": "Servicio no disponible"}, 503


    if not permitido:
        abort(403)

    aprobar_transferencia(id)
    return {"estado": "aprobada"}
```

Calidad: limitar trabajo y recursos (i)

CONCEPTO

Un **programa correcto** para **entradas pequeñas puede agotar recursos con una solicitud** enorme.

EJEMPLO / APLICACIÓN

En una plataforma industrial de monitorización de gases peligrosos, una exportación masiva puede saturar el servidor y retrasar las alarmas si comparten infraestructura.

Propón un límite y una prueba que lo compruebe.

Calidad: limitar trabajo y recursos (ii)

CONCEPTO

Un **programa correcto** para **entradas pequeñas puede agotar recursos con una solicitud** enorme.

EJEMPLO / APLICACIÓN

En una plataforma industrial de monitorización de gases peligrosos, una exportación masiva puede saturar el servidor y retrasar las alarmas si comparten infraestructura.

1. Código vulnerable

```
@app.get("/exportar")
def exportar():
    datos = db.execute(
        "SELECT * FROM mediciones"
    ).fetchall()

    return jsonify(datos)
```

Problema: si existen diez millones de mediciones, `fetchall()` intenta cargarlas todas en memoria, pudiendo provocar una denegación de servicio (DoS).

Propón un límite y una prueba que lo compruebe.

Calidad: limitar trabajo y recursos (iii)

2. Control: limitar a 10.000 registros

```
# Código Python
MAX_FILAS = 10000

@app.get("/exportar")
def exportar():
    datos = db.execute(
        "SELECT * FROM mediciones LIMIT :n",
        {"n": MAX_FILAS + 1}
    ).fetchall()

    if len(datos) > MAX_FILAS:
        return {"error": "Exportación demasiado grande"}, 422

    return jsonify(datos)
```

MAX_FILAS + 1 sirve para detectar si existen más registros de los que permitimos exportar, sin tener que recuperar millones de filas.

Calidad: limitar trabajo y recursos (iv)

Alternativa a 2. Control: limitar a 10.000 registros y conocer los restantes

```
MAX_FILAS = 10000

datos = db.execute("""
    SELECT *,
           COUNT(*) OVER () AS total
    FROM mediciones
    ORDER BY id
    LIMIT :limite
""", {"limite": MAX_FILAS}).fetchall()

total = datos[0].total if datos else 0

print("Registros totales:", total)
print("Registros recuperados:", len(datos))
```

Resultado si hubiera 10500 registros:

- Recupera 10000 registros
- Registros totales: 10500
- Registros recuperados: 10000

Encapsulación: separar datos entre usuarios (i)

El **estado compartido accidentalmente puede mezclar información de sesiones distintas**.

EJEMPLO

```
# Código Python
def registrar(usuario, vistos=[]):
    vistos.append(usuario)
    return vistos

registrar("A")
registrar("B")
```

¿La segunda llamada empieza con una lista vacía?

Encapsulación: separar datos entre usuarios (ii)

El **estado compartido accidentalmente puede mezclar información de sesiones distintas**.

EJEMPLO

```
# Código Python
def registrar(usuario, vistos=[]):
    vistos.append(usuario)
    return vistos

registrar("A")
registrar("B")
```

¿La segunda llamada empieza con una lista vacía?

En Python, una lista usada como argumento por defecto se crea una vez y se reutiliza en sucesivas llamadas.

Encapsulación: separar datos entre usuarios (iii)

¿Por qué puede ser un problema de seguridad?

EJEMPLO: Una aplicación hospitalaria utilizamos esa función para almacenar los identificadores de los historiales consultados por cada médico.

```
# Código Python
def registrar_historial(id_paciente, vistos=[]):
    vistos.append(id_paciente)
    return vistos
```

El médico A consulta el paciente 100:

```
registrar_historial(100)
# Devuelve [100]
```

El médico B consulta el paciente 200:

```
registrar_historial(200)
# Devuelve [100, 200]
```

Recibe información sobre un paciente consultado por otro médico

Encapsulación: separar datos entre usuarios (iv)

¿Cómo lo corregimos?

```
def registrar(usuario, vistos=None):
    if vistos is None:
        vistos = []

    vistos.append(usuario)
    return vistos

print(registrar("A"))  # ['A']
print(registrar("B"))  # ['B']
```

Se crea una lista nueva en cada llamada cuando no se proporciona ninguna.

No proporcionamos una lista

```
usuario = "A"
vistos = None
```

Proporcionamos una lista

```
mi_lista = ["B"]
registrar("A", mi_lista)
```

Entorno: la configuración también cuenta (i)

CONCEPTO

El **código** puede ser razonable y el despliegue exponer información que **no** debería ser pública.

EJEMPLO / APLICACIÓN

Una aplicación bancaria desplegada con el modo de depuración activado, que puede exponer información interna y, en determinadas condiciones, permitir ejecutar código en el servidor

Entorno: la configuración también cuenta (ii)

CONCEPTO

El **código** puede ser razonable y el despliegue exponer información que **no** debería ser pública.

EJEMPLO / APLICACIÓN

Una aplicación bancaria desplegada con el modo de depuración activado, que puede exponer información interna y, en determinadas condiciones, permitir ejecutar código en el servidor

```
#código Python
@app.get("/informe")
def informe():
    mes = int(request.args["mes"])
    return generar_informe(mes)

if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=True)
```

Atacante envía: `/informe?mes=abc`

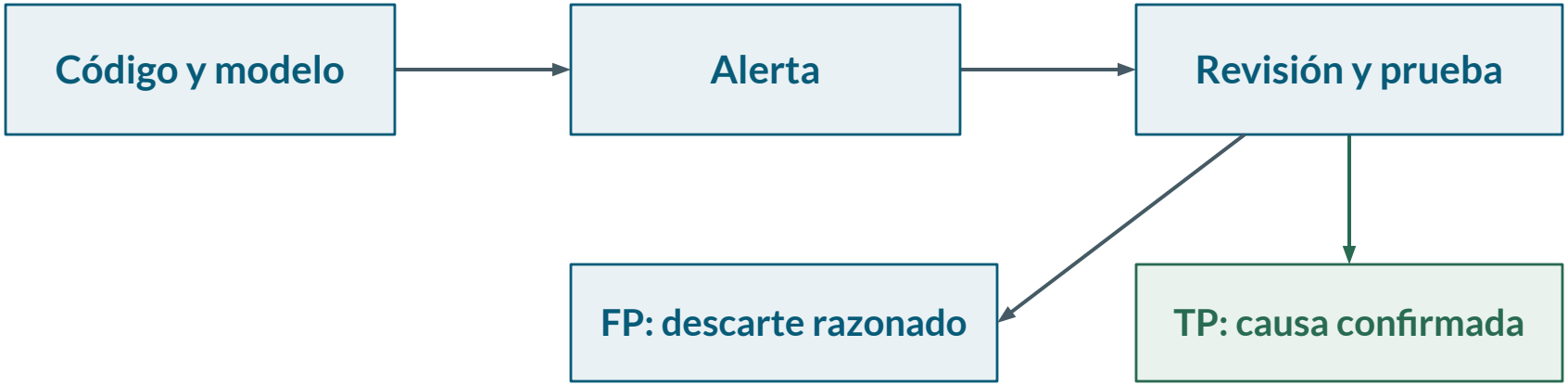
abc no se puede convertir a entero y produce la excepción `ValueError`, lo que puede exponer:

Trazas de ejecución (stack traces), Rutas internas y fragmentos del código fuente y ejecutar código Python en el servidor porque el depurador es interactivo.

SAST vs. DAST

Característica	SAST	DAST
Significado	Static Application Security Testing	Dynamic Application Security Testing
Tipo de análisis	Estático	Dinámico
¿Ejecuta la aplicación?	No	Sí
¿Necesita código fuente?	Habitualmente sí	No
¿Qué busca?	Posibles vulnerabilidades en el código	Vulnerabilidades observables durante la ejecución
Relación habitual	Caja blanca	Caja negra
Herramientas	SonarQube, Semgrep, CodeQL, etc.	OWASP ZAP, Burp Suite, etc.
Momento de aplicación	Desde el desarrollo	Cuando existe una aplicación ejecutable

SAST: una alerta es una hipótesis (i)



Falso Negativo (FN): en el contexto de herramientas SAST o DAST, significa que **existe una vulnerabilidad real en el código, pero la herramienta no la detecta y no genera ninguna alerta.**

True Positive (TP): existe vulnerabilidad y la herramienta lo detecta.

SAST: una alerta es una hipótesis (ii)

EJEMPLO: vulnerabilidad **SQL Injection**

```
#código Python
nombre = request.args["nombre"]
query = f"SELECT * FROM usuarios WHERE nombre = '{nombre}'"
db.execute(query)
```

Ana'; DELETE FROM usuarios; --

Supongamos que Semgrep o SonarQube lo analiza y no genera ninguna alerta.

Los cuatro posibles resultados

	Vulnerabilidad real	**Sin vulnerabilidad**
La herramienta genera alerta	TP — Verdadero positivo	FP — Falso positivo
La herramienta no genera alerta	FN — Falso negativo	TN — Verdadero negativo

Análisis estático - Comprobaciones de tipo

Ejemplo de **comprobación de tipo** (en java) - **falso positivo**

```
short s = 0;
int i = s; /* the type checker allows this */
short r = i; /* false positive: this will cause a type checking error at compile time */
```

No cumple con las reglas de seguridad aunque son lógicamente correctas y la salida del compilador de Java es:

```
$ javac bar.java
bar.java:12: possible loss of precision
found : int
required: short
short r = i; /* false positive: this will cause a
^
1 error
```

Análisis estático - Comprobaciones de estilo

EJEMPLO:

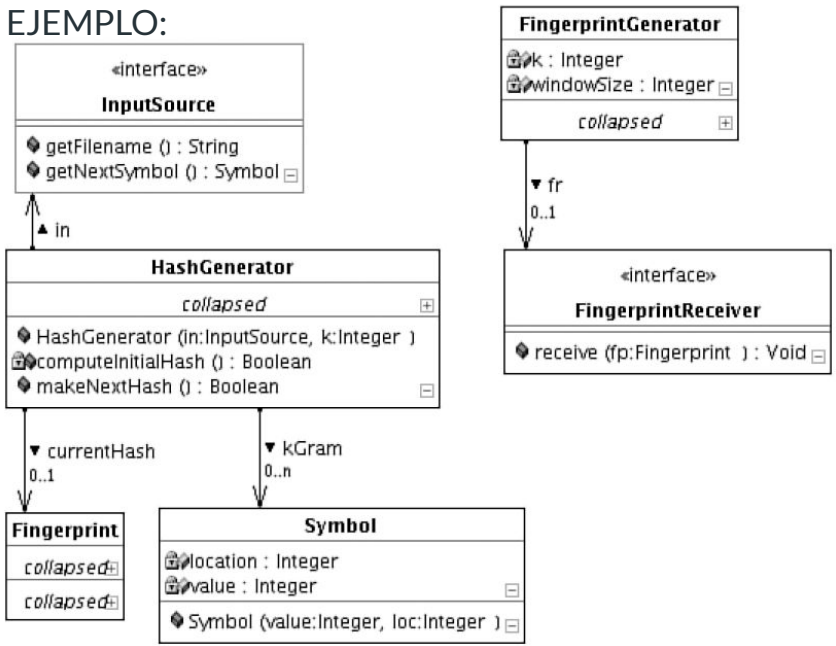
```
//código en C
int CalcularTotal(int precio,int cantidad){
int Resultado=precio*cantidad;
return Resultado;
}
```

El código es funcionalmente correcto, pero presenta problemas de estilo:

- No hay espacios después de la coma ni alrededor de =.
- La indentación es incorrecta.
- Los nombres CalcularTotal y Resultado no siguen una convención habitual de C.
    - Variables y funciones (snake_case). Ejemplo: precio_total
    - Funciones (depende del proyecto). Ejemplo: resultado (suele ser más común).
- Otras observaciones: funciones obsoletas, estructura del programa, etc.

Análisis estático - Comprensión del programa

EJEMPLO:



(Chess et al. 2007)

- Tratan de ayudar a los programadores a **comprender el funcionamiento de un programa**.
- **Refactorización automática del programa** (ej., cambiar el nombre de variables, dividir una sola función en múltiples funciones, etc.).

Análisis estático - Verificación de programas y comprobación de propiedades

- **Comprobar matemáticamente que un programa cumple una especificación**, es decir, que su comportamiento respeta los requisitos establecidos.
- **Comprobación de equivalencia** verificando que el código coincide con una especificación completa.
- **Verificar comportamiento del programa y propiedades concretas** que deban cumplirse.
- **Definir propiedades que se desean verificar.**
- **Proporcionar contraejemplos**. Cuando la herramienta detecta un incumplimiento, puede proporcionar una secuencia de operaciones que demuestra cómo se vulnera la propiedad.

Análisis estático - Verificación de programas y comprobación de propiedades

Ejemplo de **verificación de programas** (códico C)

```
1:  inBuf = (char*) malloc(bufSz);
2:  if (inBuf == NULL)
3:       return -1;
4:  outBuf = (char*) malloc(bufSz);
5:  if (outBuf == NULL)
6:       return -1;
```

Si la primera condición no se cumple y sí se cumple la segunda, se quedará memoria reservada. Contraejemplo:

```
Violation of property "allocated memory should always be freed":
line 2: inBuf != NULL
line 5: outBuf == NULL
line 6: function returns (-1) without freeing inBuf
```

Fuga de memoria (memory leak), asociada a CWE-401.
Solución: añadir `free(inBuf)` antes del segundo `return -1`

Análisis estático - Búsqueda de errores (*bugs*)

Las herramientas **incluyen reglas que describen patrones**, que señalan **puntos donde el programa se comporta de manera diferente** a lo que pretende el programador.

Ejemplo (código Java)

```
1:  if (this.fitz == null) {
2:      synchronized (this) {
3:          if (this.fitz == null) {
4:              this.fitz = new Fitzer();
5:          }
6:      }
7:  }
```

Patrón *double-checked locking* (doble comprobación con bloqueo) para asignar como máximo un objeto y minimizar el número de veces que un hilo entra en el bloque sincronizado.

```
M M DC: Possible doublecheck on Fizz.fitz in Fizz.getFitz()
At Fizz.java:[lines 1-3]
```

Habría que sincronizar todo el método

Análisis estático - Revisión de la seguridad

Buscan llamadas a determinadas funciones (ej., `strcpy()`) que a veces se usan indebidamente y deben ser inspeccionadas manualmente.

Ejemplo (código C)

```
int main(int argc, char* argv[]) {
    char buf1[1024];
    char buf2[1024];
    char* shortString = "a short string";
    strcpy(buf1, shortString); /* safe use of strcpy */
    strcpy(buf2, argv[0]); /* dangerous use of strcpy */
...
```

Inseguro porque no se controla el tamaño y se puede producir un desbordamiento de búfer.
E.j., de soluciones más seguras: strncpy(dest, src, size_t) que tiene un parámetro size_t adicional n que limita el número de caracteres a copiar.

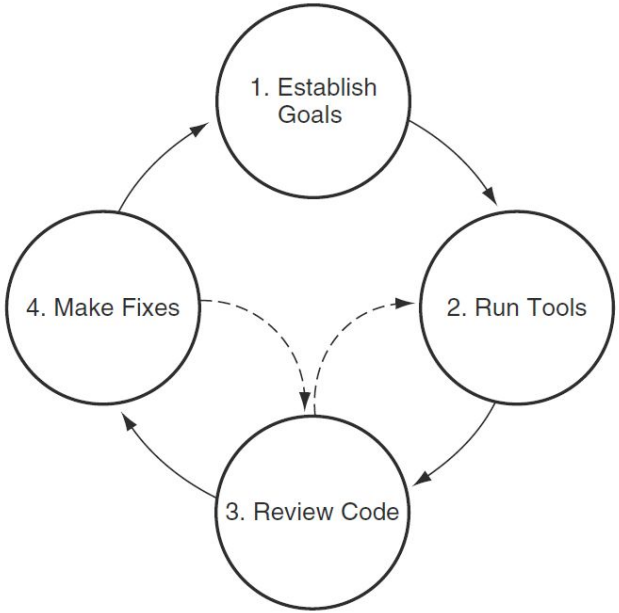
Análisis estático (i)

- Atribuir **significado a un código fuente** requiere dar sentido al **texto del programa**, comprender las **bibliotecas** en las que se basa y saber cómo encajan los diversos **componentes del programa**.
- Diferentes **compiladores** interpretan el **código** de diferentes maneras.
- Una **herramienta de análisis estático** debe conocer**:**
  - las **reglas que sigue el compilador** para analizar el código de la misma manera; y
  - los efectos de las llamadas a las **bibliotecas**, **funciones** y **métodos**;
- La **calidad de los resultados** que arroje la herramienta depende de la **calidad del modelo del programa y de las caracterizaciones de las bibliotecas**.
- Las **herramientas de análisis estático** suelen sacrificar cierta precisión para lograr una **mejor escalabilidad**.

Análisis estático (ii)

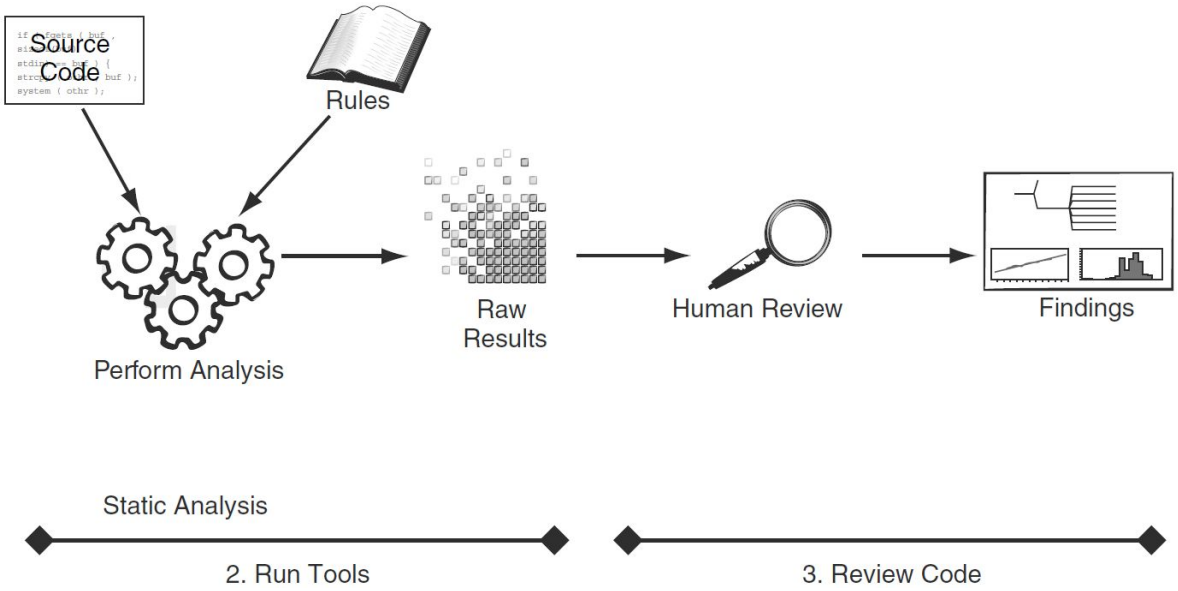
- La **velocidad** con la que las herramientas realizan el análisis del código estático es diferente.
- El **contexto de análisis** es diferente al depender del software que analiza. P. ej., fallan de forma diferente:
  - servidores y clientes;
  - sistemas operativos y aplicaciones de escritorio.
- Las **herramientas de análisis estático** deben contar con el **conjunto adecuado de defectos a buscar**.

El ciclo de revisión del código



Definir objetivos → ejecutar análisis → revisar hallazgos → corregir → volver a comprobar.

Ejecutar y conservar la evidencia



Bibliografía

Chess, B., & West, J. (2007). Secure programming with static analysis. Pearson Education.

Mead, N. R., Allen, J. H., Barnum, S., Ellison, R. J., & McGraw, G. R. (2004). Software security engineering: a guide for project managers. Addison-Wesley Professional.

¡GRACIAS POR VUESTRA ATENCIÓN!



Sara Balderas Díaz (sara.balderas@uca.es)
Gabriel Guerrero Contreras (gabriel.guerrero@uca.es)